

Defect Prediction, What-If Analysis e Refactoring Automatico via LLM

Progetto: Apache Syncope

Valentina Jin – Matricola: 0365400

Studentessa del Corso di Laurea Magistrale in Ingegneria Informatica

Università degli Studi di Roma Tor Vergata

Anno Accademico 2025/2026

Indice

1.	Introduzione	2
2.	Metodologia	2
2.1	Milestone 1: Creazione del dataset	2
2.2	Milestone 2: Valutazione dei classificatori	3
2.3	Milestone 3: What-if analysis	4
2.4	Milestone 4: Refactoring automatico	5
3.	Results	6
3.1	RQ1 – Come si caratterizza il dataset costruito per Syncope?	6
3.2	RQ2 – Quale classificatore risulta più accurato?	6
3.3	RQ3 – Quante classi buggy si sarebbero potute prevenire con zero smell?	7
3.4	RQ4 – Un refactoring generato da un LLM compila, rimuove gli smell originari, e preserva il comportamento atteso?	7
4.	Threats to validity	9
4.1	Milestone 1 – Costruzione del dataset	9
4.2	Milestone 2 – Valutazione dei classificatori	9
4.3	Milestone 3 – What-if analysis	10
4.4	Milestone 4 – Refactoring automatico	10
5.	Conclusioni	11
6.	Takeaway messages	11
	Riferimenti bibliografici	12
	Disponibilità del materiale	12
	Appendice A1: Tabelle e Figure	13
	Appendice B1: Refactor prompt per LLM	21

1. Introduzione

La previsione automatica dei difetti software (*defect prediction*) consente di indirizzare lo sforzo di testing verso le componenti più a rischio, riducendo i costi di quality assurance. Parallelamente, la diffusione dei Large Language Model come strumenti di refactoring automatico solleva una domanda ancora poco esplorata: un refactoring generato da un LLM rimuove effettivamente i code smell senza introdurre regressioni comportamentali?

Questo lavoro affronta entrambe le domande attraverso uno studio empirico su Apache Syncope, articolato in quattro milestone: costruzione di un dataset di defect prediction, valutazione di classificatori di Machine Learning, *what-if analysis* sull'assenza di smell, e generazione/validazione di refactoring automatico via LLM su due classi rappresentative del progetto (CommandLogic, NotificationDataBinderImpl). Lo studio risponde alle seguenti research question:

- **RQ1** — Come si caratterizza il dataset costruito per Syncope (dimensione, distribuzione della bugginess)?
- **RQ2** — Quale classificatore risulta più accurato nel predire le classi difettose?
- **RQ3** — Quante classi buggy si sarebbero potute prevenire in assenza di code smell?
- **RQ4** — Un refactoring generato da un LLM compila, rimuove gli smell originari, e preserva il comportamento atteso delle classi?

Il documento è organizzato come segue: la Sezione 2 descrive la metodologia per ciascuna milestone; la Sezione 3 risponde alle RQ; la Sezione 4 discute le minacce alla validità; le Sezioni 5 e 6 raccolgono conclusioni e takeaway messages.

2. Metodologia

2.1 Milestone 1: Creazione del dataset

Il dataset è definito a livello classe-release: ogni istanza rappresenta l'osservazione di una classe Java di produzione in una specifica release di Apache Syncope, identificata dalla coppia (ClassName, Release), e associa a tale osservazione 14 metriche software, il conteggio degli smell e la label finale di bugginess (yes/no). La pipeline è organizzata in due fasi eseguite da due entry point distinti: LabelingMain.java, che produce l'inventario etichettata e i fix commit, e Milestone1DatasetMain.java, che calcola le metriche e produce il dataset finale.

2.1.1 Selezione delle release e snapshot

Delle 81 release totali disponibili nel repository di Apache Syncope, sono state selezionate le prime 25 in ordine cronologico (**Tabella 1**), limitando l'osservazione alle release più antiche per garantire un labeling più stabile e ridurre il rischio di *snoring* (classi etichettate erroneamente come non buggy perché i loro difetti non sono ancora stati scoperti). Per ciascuna release selezionata, si è utilizzato un ID numerico incrementale (1-25) al posto del nome semantico della versione, per semplificare i calcoli successivi legati alla Proportion. Lo snapshot del codice è stato risolto tramite i **tag Git ufficiali** (syncope-X.Y.Z), non tramite ricerca dell'ultimo commit antecedente alla data di release su master. Questa scelta è stata motivata da un problema riscontrato empiricamente: alcune patch release derivano da branch di manutenzione paralleli e non da master; un approccio basato sulla data avrebbe incluso fino a 700 classi mai realmente rilasciate in quella versione ("classi fantasma"). Da ciascuno snapshot sono state estratte esclusivamente le classi Java di produzione, escludendo i file di test.

2.1.2 Ticket, Proportion e labeling

I ticket sono stati letti dal tracker Jira del progetto, considerando validi i difetti di tipo Bug con risoluzione Fixed. Per ogni ticket sono state ricostruite Opening Version (**OV**, release alla data di apertura), Fixed Version (**FV**, release alla data di risoluzione) e Injected Version (**IV**, release di introduzione del difetto). La IV è stata ricavata dalla prima Affected Version dichiarata quando disponibile e coerente ($\leq OV, < FV$); in assenza di Affected Version affidabili, è stata stimata tramite **Proportion Total**, applicando il moltiplicatore medio P calcolato sui ticket con AV nota alla formula $IV = FV - P \cdot (FV - OV)$. Il linkage ticket-commit è stato effettuato con `git log --all`, ossia su tutti i branch e non solo su master: questa scelta ha corretto un secondo problema riscontrato empiricamente, poiché limitarsi a master faceva perdere i fix committati sui branch di manutenzione, portando i fix commit individuati da 1.059 a 1.854.

L'etichettatura delle classi come buggy nell'intervallo [IV, FV) è stata infine assegnata tramite **SZZ**, usando il 100% delle release disponibili per il linkage (non solo le 25 selezionate), poiché un bug introdotto in una release antica può essere scoperto e corretto solo molte release più avanti.

2.1.3 Calcolo delle metriche

La scelta delle metriche ha seguito il criterio delle tre dimensioni classiche indicate a lezione: metriche di codice (dimensione e complessità strutturale della classe), metriche di processo (attività e instabilità osservate nella storia dei commit) e fattori umani (dispersione dell'ownership tra gli sviluppatori). Sono state selezionate 14 metriche complessive più il conteggio degli smell, riportate in **Tabella 2**, evitando di includere coppie di metriche ridondanti tra loro: dimensione e complessità, ad esempio, sono spesso fortemente correlate, ma sono state mantenute entrambe perché catturano aspetti concettualmente distinti — la prima la mole di codice, la seconda la difficoltà del flusso di controllo.

Una decisione metodologica esplicita ha riguardato la finestra temporale di calcolo. Come indicato a lezione, ogni metrica di processo può in linea di principio essere calcolata in due declinazioni: cumulativa, a partire dalla prima comparsa storica della classe nel repository, e specifica, limitata alla sola release corrente. Si è scelto di **non raddoppiare** le metriche in queste due varianti, calcolando ciascuna una sola volta nella finestra della release corrente. La motivazione richiama direttamente uno degli argomenti discussi a lezione a proposito della feature selection: al di là del costo computazionale di addestramento — irrilevante, poiché il training avviene offline — il costo realmente significativo in un contesto ingegneristico è quello di **sviluppo degli estrattori** che calcolano le metriche a partire dai dati grezzi di Git. Raddoppiare ogni metrica di processo avrebbe raddoppiato anche la logica di estrazione da sviluppare e verificare, in una pipeline già resa complessa dalla risoluzione degli snapshot tramite tag Git e dal linkage esteso a tutti i branch di manutenzione, a fronte di un beneficio predittivo atteso limitato, dato che le due declinazioni di una stessa metrica di processo tendono a essere fortemente correlate tra loro. Questa scelta, insieme alle sue implicazioni sulla capacità del dataset di distinguere segnali storici da segnali recenti, è discussa come limite metodologico in Sezione 4.

2.1.4 Code smell

Il numero di smell (NSmells) per classe è stato calcolato tramite **PMD** con ruleset *quickstart.xml*. Poiché PMD richiede file su disco e non legge direttamente dagli oggetti Git, ogni snapshot è stato materializzato tramite *git worktree* prima dell'analisi statica.

2.1.5 Merge e validazione

Le metriche, gli smell e le label sono stati uniti in un unico CSV finale (*dataset_SYNCOPE_final.csv*): 16.063 righe, 19 colonne (progetto, release, nome classe, 14 metriche, NSmells, bugginess), con 1.054 istanze buggy pari al 6,56%. Il dataset è stato validato tramite controlli aritmetici esaustivi tra metriche correlate (ad es. $LOC_{Touched} = 2 \times LOC_{Added} - Churn$), riscontrando zero violazioni su tutte le righe.

2.2 Milestone 2: Valutazione dei classificatori

2.2.1 Preparazione dei dati e strumenti

Il dataset prodotto nella Milestone 1 è stato convertito in formato ARFF e caricato in **Weka**. I due moduli GUI di Weka sono stati usati con ruoli distinti:

- **Weka Explorer** è stato usato esclusivamente per attività esplorative qualitative: valutare visivamente lo sbilanciamento della classe target (**Figura 1**), confrontare rapidamente configurazioni alternative di feature selection (es. l'output di CfsSubsetEval+BestFirst rispetto a InfoGain+Ranker) prima di decidere quale adottare, e verificare singolarmente il comportamento dei filtri di balancing su una prova isolata.
- **Weka Experimenter** è stato usato per produrre i risultati ufficiali riportati nel report: nella tab Setup sono state configurate le 12 combinazioni sperimentali (3 classificatori $\times$ 4 configurazioni di filtri) con 10 ripetizioni di 10-fold cross-validation ciascuna; la tab Run ha eseguito i 1.200 fold complessivi; la tab Analyse ha prodotto l'export in formato CSV (*weka_results_summary.csv*), successivamente elaborato da un modulo Java custom per il calcolo di NPofB20 (non nativo in Weka) e per l'aggregazione finale in **Tabella 4**.

Per evitare l'errore metodologico segnalato esplicitamente a lezione – applicare un filtro di feature selection sull'intero file tramite la tab Preprocess dell'Explorer prima della cross-validation, che equivale a far "sbirciare" il test set all'algoritmo di selezione – feature selection e balancing non sono mai stati applicati come filtri globali. Sono stati invece incapsulati in un *FilteredClassifier*, così da essere ricalcolati esclusivamente sul training set di ciascun fold sia in Explorer (nelle prove esplorative) sia in Experimenter (nella run ufficiale).

2.2.2 Classificatori e validazione

Sono stati valutati i tre classificatori richiesti: **RandomForest**, **NaiveBayes** e **IBk**. La valutazione è stata eseguita tramite **10 times 10-fold cross-validation** (10 ripetizioni indipendenti di una 10-fold CV, per un totale di 100 valutazioni per configurazione), per ridurre la dipendenza dei risultati da una singola partizione casuale del dataset.

2.2.3 Feature selection e balancing

Per la **feature selection** è stata adottata la combinazione **InfoGainAttributeEval** + **Ranker**, selezionando le prime 6 feature per potere informativo rispetto alla classe target. Questa configurazione è stata preferita rispetto a due alternative testate: CfsSubsetEval + BestFirst, che scartava sistematicamente NFix – una metrica di processo con forte potere predittivo individuale – e Wrapper + GreedyStepwise, che collassava la selezione a una singola feature, risultando eccessivamente aggressivo.

Per il **balancing** è stato adottato il filtro **Resample** in modalità oversampling (*biasToUniformClass=1.0*), con sampleSizePercent calcolato analiticamente a partire dal tasso di bugginess reale del dataset, in modo da produrre un training set effettivamente bilanciato tra le due classi. Sono state scartate due alternative: SpreadSubsample (undersampling), giudicato eccessivamente distruttivo per la quantità di informazione scartata, e SMOTE, escluso per il costo computazionale delle interpolazioni sintetiche.

Nelle configurazioni in cui feature selection e balancing sono stati applicati congiuntamente, i due filtri sono stati incapsulati in una pipeline *MultiFilter*, applicando sempre la feature selection prima del balancing, e l'intera pipeline è stata incapsulata nel classificatore tramite *FilteredClassifier* di Weka, così da applicare i filtri esclusivamente sui fold di training durante la cross-validation ed evitare data leakage.

2.2.4 Configurazioni sperimentali e metriche

Ogni classificatore è stato valutato su quattro configurazioni progressive: baseline senza filtri, solo feature selection, solo balancing, e combinazione di entrambe. Le prestazioni sono state misurate tramite **Precision**, **Recall**, **AUC** e **Kappa**, tutte calcolate rispetto alla classe positiva yes (buggy).

È stata inoltre calcolata **NPofB20**, una metrica *effort-aware* che misura la percentuale di classi buggy individuabili ispezionando solo il 20% delle righe di codice a maggior rischio predetto, normalizzando così il confronto rispetto alla dimensione delle classi. Poiché Weka non produce questa metrica nativamente, è stata reimplementata programmaticamente tramite l'API Java di Weka, ripetendo 10 volte la 10-fold cross-validation per ottenere una distribuzione di 10 valori per ciascuna configurazione, anziché un singolo valore puntuale.

2.3 Milestone 3: What-if analysis

La Milestone 3 ha stimato quante istanze classificate come buggy nello scenario reale sarebbero state classificate come non buggy in uno scenario ipotetico privo di code smell, utilizzando come classificatore di riferimento (*BClassifier*) la configurazione **Baseline RandomForest** selezionata in Milestone 2.

A partire dal dataset finale della Milestone 1 sono stati costruiti quattro sottoinsiemi:

- **A**: il dataset completo;
- **B+**: le istanze con almeno uno smell ($NSmells > 0$);
- **C**: le istanze senza smell ($NSmells = 0$), tale che $A = B+ \cup C$;
- **B**: copia sintetica di B+, con NSmells forzato a 0 su ogni istanza.

La partizione è stata effettuata in Excel, poi i quattro sottoinsiemi sono stati convertiti in formato ARFF.

Il classificatore è stato allenato una sola volta sull'intero dataset A, e successivamente valutato separatamente sui quattro sottoinsiemi tramite l'opzione "Supply test set" di Weka Explorer, che consente di applicare un modello già addestrato a un test set esterno senza ri-addestrarlo. Per ciascun sottoinsieme è stata considerata la somma delle istanze predette come buggy ("predicted Yes"), oltre al conteggio delle predizioni, è stata registrata anche la matrice di confusione completa per un'analisi più approfondita in Sezione 3.3: l'obiettivo dell'analisi è infatti confrontare il comportamento del modello prima e dopo la rimozione simulata degli smell, non valutarne l'accuratezza.

A partire dai quattro conteggi (buggy predette su A, B+, C, B) sono state calcolate tre metriche riassuntive, secondo le formule indicate a lezione:

- **In Total** = predette buggy su B+ – predette buggy su B;
- **In Proportion** = In Total / predette buggy su A;
- **Out of preventable** = In Total / predette buggy su B+.

I risultati numerici e la loro discussione, incluso un approfondimento diagnostico condotto per verificarne la plausibilità, sono riportati in Sezione 3.3.

2.4 Milestone 4: Refactoring automatico

2.4.1 Selezione delle classi

Le classi oggetto di refactoring sono state selezionate a partire dall'ultima release del progetto, applicando un ranking di tutte le classi per numero di smell (*NSmells*) rilevati da PMD. Prima del ranking sono stati applicati alcuni filtri per escludere classi non rappresentative: solo classi nel package core/ (logica di business, esclusi moduli di infrastruttura/test), $LOC \geq 60$, $5 \leq CC \leq 70$, almeno 2 metodi pubblici sostanziali (parametri non banali, almeno 10 righe, $CC > 1$, per escludere getter/setter) e *NSmells* > 0. Questi criteri sono stati scelti e documentati per garantire che le classi selezionate offrissero materiale sufficiente per un refactoring significativo e per le successive fasi di testing.

Dalla lista ordinata risultante, le due classi sono state selezionate seguendo la regola posizionale indicata a lezione.

Sono risultate selezionate **CommandLogic** (rank 3, *NSmells*=6, modulo core/idrepo/logic) e

NotificationDataBinderImpl (rank 29, *NSmells*=1, modulo core/provisioning-java) – rispettivamente una classe con smell numerosi e una quasi priva di smell.

2.4.2 Generazione delle varianti

Per ciascuna classe sono state generate quattro varianti di refactoring (C1-C4) a partire dalla versione originale (C0), utilizzando **Microsoft Copilot** in modalità **Think Deeper**. Il codice di C0 è stato fornito come file allegato, non incollato come testo nel prompt, per evitare errori di elaborazione su classi di dimensioni non trascurabili.

I prompt utilizzati, riportati integralmente in **Appendice B1**, hanno la struttura articolata in cinque punti: definizione del ruolo (sviluppatore Java esperto), richiesta di rimuovere gli smell diagnosticati da SonarCloud senza alterare la funzionalità di C0, richiesta di superare i test forniti in input, vincolo esplicito di non introdurre modifiche non richieste (la variante deve poter sostituire C0 come *drop-in replacement*, mantenendo package, nome classe e firme pubbliche), e un'istruzione finale di accuratezza ("prenditi tutto il tempo necessario").

Le quattro varianti si distinguono per il contesto progressivamente più ricco fornito in input, mantenendo costante il prompt base:

- **C1** — solo C0 e la diagnostica SonarCloud degli smell da rimuovere, nessun test;
- **C2** — in aggiunta, le suite di test Black Box;
- **C3** — in aggiunta, le suite di test Control Flow / White Box;
- **C4** — in aggiunta, le suite di test basate su Mutation Testing.

Questa progressione permette di osservare se un contesto sperimentale più ricco produca varianti più conservative e maggiormente aderenti al comportamento di C0, o se converga rapidamente su un unico output indipendentemente dai test forniti.

2.4.3 Valutazione delle varianti

Ogni variante è stata valutata rispondendo a quattro domande, indicate come criterio di analisi dei risultati:

1. C_X compila, da sola e nel contesto del modulo Maven reale?
2. C_X presenta smell (nessuno, gli stessi di C0, o nuovi)?
3. Una feature nota per essere positivamente correlata alla bugginess nel dataset di Milestone 1 (Sezione 3.1, **Tabella 3**) risulta più alta in C_X rispetto a C0?
4. Una feature negativamente correlata alla bugginess risulta più alta in C_X rispetto a C0?

Le prime due domande sono state verificate tramite compilazione reale nel modulo Maven del progetto e scansione SonarCloud; le ultime due tramite ricalcolo delle metriche strutturali (SIZE_LOC, NOM, AVG_METHOD_SIZE, CyclomaticComplexity, FanOut) su ciascuna variante, confrontate con le correlazioni già stabilite in Milestone 1.

Tecnicamente, per ciascuna variante C_X la classe originale è stata **temporaneamente sostituita** con la versione generata all'interno del modulo Maven reale (*core/idrepo/logic* per *CommandLogic*, *core/provisioning-java* per *NotificationDataBinderImpl*), per poi lanciare *mvn compile* (per la verifica di compilazione) e successivamente *mvn test* sulle suite di regressione disponibili, con i flag *-Dmaven.build.cache.enabled=false* per evitare che la build cache, basata sui checksum dei sorgenti, saltasse silenziosamente la ricompilazione o la riesecuzione dei test a fronte di una classe sostituita. Al termine della verifica su ciascuna variante, il file originale è stato ripristinato prima di procedere con la variante successiva, per non alterare lo stato del repository.

Per verificare la preservazione del comportamento osservabile, ogni variante compilabile è stata sottoposta alle suite di regressione già disponibili dal track di testing (manuali BB/WB/MT, Randoop, LLM). Per evitare il bias di validare un refactoring generato da un LLM esclusivamente con test generati dallo stesso LLM, la validazione ha incluso sistematicamente anche le suite scritte manualmente, così da disporre di una ground truth indipendente dal modello.

EvoSuite, la quarta tecnica di generazione automatica prevista dal corso, non è stata inclusa: la versione bytecode richiesta dal progetto (Java 25, major version 69) risulta incompatibile con l'ASM utilizzato internamente da EvoSuite, che legge le classi della libreria standard direttamente dalla JVM in esecuzione e non solo dal classpath fornito – un limite non aggirabile senza declassare la JVM dell'intero ambiente di sviluppo.

3. Results

3.1 RQ1 – Come si caratterizza il dataset costruito per Syncope?

Il dataset finale contiene 16.063 istanze classe-release, di cui 1.054 etichettate come buggy (6,56%), a conferma di un forte sbilanciamento verso la classe non difettosa tipico di questo tipo di studi. La distribuzione per release, già presentata in **Tabella 2**, mostra un tasso di bugginess massimo alla release 1.0.5 (18,74%) e minimo alla release 1.2.3 (2,07%), senza un trend monotono nel tempo.

Per quanto riguarda gli smell, 6.079 istanze (37,8%) presentano almeno uno smell rilevato da PMD, mentre le restanti 9.984 (62,2%) ne sono prive; il numero massimo di smell osservato su una singola istanza è 44.

La **Tabella 3** (in appendice) riporta la **correlazione** di ciascuna metrica con la label Bugginess, calcolata sia come Pearson point-biserial sia come Spearman. Le associazioni più marcate riguardano FanOut ($\rho=0,259$, $r=0,325$), CyclomaticComplexity ($\rho=0,230$, $r=0,265$) e NOM ($\rho=0,192$, $r=0,175$), tutte statisticamente significative ($p<0,001$): le classi più grandi, più complesse e più accoppiate risultano sistematicamente più difettose. NAuth e NSmells mostrano correlazioni moderate ma comunque significative ($\rho\approx 0,14-0,15$). Le metriche di processo legate al volume delle modifiche (LOCtouched, LOCadded, Churn, ChangeSetSize) mostrano invece correlazioni deboli o trascurabili in questo dataset, mentre Age mostra una lieve correlazione negativa ($\rho=-0,032$): le classi più "giovani" tendono a essere leggermente più difettose.

3.2 RQ2 – Quale classificatore risulta più accurato?

Sono stati confrontati RandomForest, NaiveBayes e IBk su quattro configurazioni (baseline, solo feature selection, solo balancing, combinazione delle due), valutati tramite **10 times 10-fold cross-validation**, con Precision, Recall, F-Measure, AUC e Kappa calcolate sulla classe positiva *yes*, integrate da NPofB20 come misura *effort-aware*. I risultati completi sono riportati in **Tabella 4**.

La configurazione complessivamente migliore è **RandomForest in configurazione baseline**, senza filtri: Precision=0,713, F-Measure=0,547, AUC=0,886, Kappa=0,523, NPofB20=63,56%. Questa configurazione domina su cinque delle sei metriche misurate. L'unica eccezione è la Recall, dove le configurazioni con balancing risultano superiori – massima per IBk con FS+Balancing (0,610) e RandomForest con FS+Balancing (0,603) – a fronte però di un crollo della Precision (rispettivamente 0,252 e 0,252, contro 0,713 della baseline). Questo comportamento è coerente con l'effetto atteso del balancing: forzando il modello a prestare più attenzione alla classe minoritaria, aumenta la capacità di individuare le istanze buggy (Recall) al costo di un numero molto più alto di falsi positivi (Precision). I boxplot in **Figura 2-7** mostrano questo pattern sull'intera distribuzione delle 100 run, non solo sulla media: la baseline RandomForest mantiene una AUC concentrata sopra 0,85 con dispersione contenuta, mentre la Recall più alta delle configurazioni bilanciate emerge chiaramente dai boxplot al prezzo di una Precision visibilmente più bassa e dispersa.

NaiveBayes è risultato sistematicamente il classificatore meno accurato in ogni configurazione e su quasi ogni metrica, coerentemente con la forte correlazione osservata tra diverse metriche del dataset (Sezione 3.1, **Tabella 3**), che viola l'assunzione di indipendenza condizionata alla base del modello – evidente anche visivamente nel boxplot di NPofB20 tra le **Figura 2-7**, dove NaiveBayes resta sotto il 25% in ogni configurazione contro il 55-65% di RandomForest e IBk.

3.3 RQ3 – Quante classi buggy si sarebbero potute prevenire con zero smell?

L'analisi what-if, condotta con RandomForest baseline (BClassifier selezionato in Sezione 3.2), ha confrontato le predizioni sui quattro sottoinsiemi A, B+, C, B definiti in Sezione 2.3. La **Tabella 5** riporta la matrice di confusione completa del BClassifier su ciascun sottoinsieme.

Un dato interessante emerge dal confronto tra B+ e B che il solo conteggio delle predizioni buggy non mostrerebbe: in B+ il modello ha FN=2 (quasi tutte le istanze realmente buggy vengono correttamente identificate, coerentemente con l'alto NPofB20 di RandomForest, Sezione 3.2), mentre nello scenario sintetico B i falsi negativi salgono a **403** – il modello smette di riconoscere come buggy istanze che, azzerato solo NSmells, restano invariate su tutte le altre metriche (**Tabella 7**). Questo salto nei falsi negativi tra B+ e B è, in altri termini, la stessa evidenza numerica dietro l'In Total=392, vista però dal lato delle etichette reali anziché delle sole predizioni.

A partire dai conteggi di "Predetti Buggy" sono state calcolate le tre metriche riassuntive già introdotte in Sezione 2.3 (In Total, In Proportion, Out of preventable): 392, 37,95%, 64,58%. Questo valore è risultato sensibilmente più alto di quanto atteso per un effetto marginale della sola rimozione degli smell. Per verificarne la plausibilità, la stessa procedura è stata ripetuta anche con **NaiveBayes** come classificatore di controllo (**Tabella 6**): il risultato (In Proportion=6,62%, Out of preventable=8,11%) è risultato molto più contenuto. Un'osservazione emersa dalla matrice di correlazione estesa (**Tabella 7**) aiuta a spiegare questo divario: **NSmells** è fortemente co-correlata con **CyclomaticComplexity** (Pearson $r=0,53$ e Spearman $\rho=0,438$, entrambe fortemente significative ($p<0,001$)). Poiché RandomForest, a differenza di NaiveBayes, può sfruttare interazioni non lineari tra feature correlate, è plausibile che il modello stia in parte "recuperando" il segnale di NSmells anche attraverso CyclomaticComplexity – feature che nello scenario what-if non viene modificata insieme a NSmells. Questa lettura resta comunque un'evidenza indiretta: non è stato condotto un esperimento controllato che la dimostri in modo conclusivo.

Il BClassifier ufficiale per la Milestone 3 resta RandomForest, coerentemente con la scelta stabilita in Sezione 3.2; NaiveBayes ha avuto qui esclusivamente un ruolo diagnostico e non sostituisce la stima principale.

3.4 RQ4 – Un refactoring generato da un LLM compila, rimuove gli smell originari, e preserva il comportamento atteso?

Ciascuna variante è stata verificata sostituendo temporaneamente il file della classe originale con la versione generata all'interno del modulo Maven reale del progetto, come descritto in Sezione 2.4.3.

Compilazione

- *NotificationDataBinderImpl*: tutte e 4 le varianti compilano correttamente.
- *CommandLogic*: C1, C2 e C4 compilano; **C3 fallisce con 6 errori** di build reale:
 - rimozione indebita di un parametro domain in due punti;
 - un tipo generico reso esplicito in modo incompatibile;

- un simbolo enum inesistente;
 - un tipo di ritorno incompatibile nella firma ereditata.
- Di questi 6 errori, **4 erano stati anticipati** da un'analisi statica preliminare; i restanti **2 sono emersi solo dalla compilazione effettiva** – a conferma che l'analisi statica riduce ma non elimina la necessità della verifica empirica.

Rimozione degli smell e maintainability

Un primo risultato trasversale a entrambe le classi è che C1 e C4 sono byte-identiche, nonostante C4 riceva molto più contesto in input (BB+CF+Mutation) rispetto a C1 (nessun test): l'informazione aggiuntiva non ha alterato l'output del refactoring.

Su *CommandLogic*, SonarCloud aveva diagnosticato 5 occorrenze dello stesso smell in C0 (regola java22, "Replace 'e' with an unnamed pattern"), di cui solo 2 riguardavano un'eccezione realmente inutilizzata; nelle restanti 3, e era effettivamente usata per il logging o per il messaggio d'errore restituito al chiamante. Il prompt chiedeva di rimuovere tutti gli smell diagnosticati da SonarCloud, e le quattro varianti hanno interpretato questa istruzione in due modi opposti: C1 e C4 l'hanno applicata in modo selettivo, correggendo solo i 2 catch dove e era davvero inutilizzata (3 smell residui, **Tabella 8**); C2 e C3 l'hanno invece eseguita alla lettera, rimuovendo tutti e 5 gli smell segnalati senza distinguere caso per caso se l'eccezione fosse effettivamente utilizzata altrove nel blocco (0 smell residui). Questa esecuzione letterale è corretta dal punto di vista della diagnostica statica, ma ha eliminato anche l'uso legittimo dell'eccezione nei 3 catch dove serviva – la causa diretta della regressione comportamentale discussa più avanti.

Azzerare gli smell originari non equivale però, in questi due casi, a un miglioramento netto della qualità del codice: sia C2 sia C3 introducono ciascuno un nuovo smell di categoria diversa, sostituendo il problema originario con uno nuovo – un esito che una lettura superficiale della sola colonna "Smell residui" in **Tabella 8** non farebbe emergere. C2 genera 2 nuove issue "clumsy" ("Immediately throw this expression instead of assigning it to the temporary variable"), conseguenza diretta della cancellazione della riga intermedia `sce.getElements().add(e.getMessage())`. C3 introduce invece 1 issue Critical, rilevata tramite una scansione SonarCloud isolata resa necessaria dal mancato compile del modulo completo: un metodo transazionale invocato tramite self-invocation (`this.read(...)`) anziché tramite dependency iniettata – un anti-pattern Spring reale, poiché la self-invocation bypassa il proxy AOP che implementa `@Transactional`.

Su *NotificationDataBinderImpl*, SonarCloud aveva diagnosticato un solo smell: un riferimento del logger all'interfaccia anziché alla classe concreta (L44, categoria Consistency, severità Minor — "Update this logger to use 'NotificationDataBinderImpl.class'"). Tutte e quattro le varianti lo correggono, azzerando gli smell residui (0, **Tabella 8**): qui la correzione è univoca, sicura e priva di qualunque rischio comportamentale, trattandosi di un riferimento statico nella dichiarazione del logger, indipendente dalla logica applicativa della classe.

La **Tabella 9** riporta la variazione delle metriche strutturali di ciascuna variante rispetto alla classe C0 originale. Anche su questo fronte, *CommandLogic_C3* è l'unica variante che si discosta in modo significativo: dimensione 156→172 LOC, dimensione media dei metodi 39,0→43,0, complessità ciclomatica 12→18. **CyclomaticComplexity** è una delle metriche più fortemente correlate alla bugginess nel dataset (Sezione 3.1, $r=0,265$, significativa): C3 ha quindi peggiorato una metrica nota per predire difettosità, oltre a non compilare.

Preservazione del comportamento

Ogni variante compilabile è stata sottoposta a tutte le suite di regressione disponibili dal track di testing (manuale BB+WB+MT, Randoop, LLM Zero-shot+Few-shot+ToT).

- **CommandLogic (Tabella 10)**: C1 e C4 superano tutti i 58 test. **C2 fallisce 4 test su 58**, tutti appartenenti alle suite LLM Few-shot e Zero-shot.
 - Causa: C2 rimuove `sce.getElements().add(e.getMessage())` anche nei catch dove `e` era effettivamente usata.
 - Nessun test manuale o Randoop asserisce sul contenuto di `getElements()` in quei percorsi; due test Few-shot e due Zero-shot lo fanno, rilevando la regressione.
 - C4 genera lo stesso identico codice di C1: eredita quindi lo stesso esito, 58/58.

- **NotificationDataBinderImpl (Tabella 11):** C1 supera tutti i 61 test. C2, C3 e C4 generano codice sorgente sostanzialmente identico a C1 (nessuna riga di logica differente), e di conseguenza lo stesso esito: nessuna regressione.

In sintesi, questo risultato evidenzia un limite della sola validazione tramite test progettati a priori (manuale, Randoop) nel rilevare regressioni introdotte da un refactoring automatico – solo l'introduzione di suite generate da LLM con tecniche diverse ha permesso di scoprire il problema di C2. Approfondito come limite metodologico in Sezione 4.

4. Threats to validity

4.1 Milestone 1 – Costruzione del dataset

- **Stima della Injected Version.** Quando la Affected Version non è disponibile o non è affidabile, la IV è stimata tramite **Proportion Total**, un moltiplicatore medio calcolato sui soli ticket con AV nota. È una stima statistica, non un dato osservato: introduce un margine di errore nell'etichettatura di bugginess che si propaga a valle su tutte le milestone successive.
- **Selezione delle sole prime 25 release su 81 totali.** La decisione di limitare l'osservazione al 30,9% più antico delle release disponibili, anziché coprire l'intera storia del progetto, è stata presa per ridurre il rischio di *snoring* (una release recente rende più probabile che i suoi difetti non siano ancora stati scoperti al momento dell'estrazione del dataset). Questa scelta comporta però un trade-off esplicito: un dataset più piccolo (16.063 istanze anziché una quantità proporzionalmente maggiore se si fossero incluse tutte le 81 release) e una minore rappresentatività delle fasi più recenti del progetto, dove pratiche di sviluppo, dipendenze e stile del codice potrebbero differire sensibilmente da quelle osservate nelle release più antiche.
- **Rischio di *snoring* residuo.** Anche con la sola selezione delle prime 25 release, il rischio non è eliminato del tutto: un bug introdotto nella release 25 potrebbe essere scoperto solo dopo la release 81, restando quindi erroneamente non rilevato nel dataset.
- **Nessuna doppia declinazione delle metriche di processo.** Come motivato in Sezione 2.1.3, si è scelto di calcolare ogni metrica di processo una sola volta, nella finestra della release corrente, anziché anche cumulativamente dall'inizio del progetto. La decisione richiama l'argomento del costo di sviluppo degli estrattori discusso a lezione a proposito della feature selection: raddoppiare ogni metrica avrebbe raddoppiato la logica di estrazione da sviluppare e verificare, a fronte di un beneficio predittivo atteso limitato, dato che le due declinazioni di una stessa metrica tendono a essere fortemente correlate tra loro. Si tratta quindi di una scelta deliberata, ma resta un limite reale: il dataset perde la capacità di distinguere se l'instabilità di una classe sia recente o storica, un'informazione che un classificatore avrebbe potuto sfruttare per discriminare meglio tra classi stabilizzate e classi ancora in fase di assestamento.
- **Scelta soggettiva delle metriche e del ruleset PMD.** Le 14 metriche selezionate e il ruleset *quickstart.xml* usato per NSmells riflettono scelte progettuali non uniche; un set diverso di metriche o un ruleset PMD più esteso avrebbe potuto produrre un dataset con potere predittivo differente.

4.2 Milestone 2 – Valutazione dei classificatori

- **Scelta della configurazione di feature selection e balancing.** InfoGain+Ranker (top 6) e Resample oversampling sono state preferite rispetto ad alternative scartate (CfsSubsetEval+BestFirst, Wrapper+GreedyStepwise, SpreadSubsample, SMOTE) sulla base di un'analisi qualitativa dei risultati intermedi, non di un criterio automatico univoco: un'esplorazione più ampia dello spazio di configurazione avrebbe potuto restituire una combinazione ancora migliore. La configurazione InfoGain+Ranker con soglia fissata a 6 feature non è stata scelta in modo arbitrario. Prima di fissare la soglia, è stata eseguita una prova preliminare con CfsSubsetEval+BestFirst – approccio Wrapper-like che, applicando la propria euristica di ricerca sui sottoinsiemi, individua autonomamente un numero di feature "ottimale" senza bisogno di una soglia imposta dall'esterno. Questa prova preliminare ha restituito un sottoinsieme di 6 feature, quindi il valore emerso da questa esplorazione è stato quindi riutilizzato come soglia per Ranker, applicato però secondo il

ranking di Information Gain. La scelta nasce da un confronto empirico, ma resta basata su un singolo test esplorativo, non su una ricerca sistematica tra più soglie (top 4, top 8, top 10...).

- **Variabilità stocastica.** Come mostrato dai boxplot (**Figura 2-7**), le prestazioni di ciascuna configurazione oscillano in modo non trascurabile tra le 100 run di cross-validation: i confronti tra configurazioni sono basati su medie, che possono nascondere sovrapposizioni parziali tra le distribuzioni.
- **Metrica NPofB20 reimplementata manualmente.** Non essendo nativa in Weka, NPofB20 è stata calcolata con un modulo Java custom che ripete la 10-fold CV tramite l'API weka-stable: un'implementazione indipendente non validata rispetto a un tool di riferimento esterno introduce un rischio residuo di errore, per quanto la coerenza dei valori ottenuti con l'andamento delle altre metriche (Precision, Recall) ne supporti la plausibilità.

4.3 Milestone 3 – What-if analysis

- **Interazione tra NSmells e altre metriche.** Il valore di In Proportion ottenuto con RandomForest (37,95%) è più alto di quanto atteso per un effetto marginale della sola rimozione degli smell. Un'osservazione emersa dalla matrice di correlazione estesa (Sezione 3.3, **Tabella 7**) aiuta a spiegarlo: NSmells è fortemente co-correlata con CyclomaticComplexity ($r=0,53$). Poiché RandomForest, a differenza di NaiveBayes, può sfruttare interazioni non lineari tra feature correlate, è plausibile che il modello stia in parte "recuperando" il segnale di NSmells anche attraverso CyclomaticComplexity – feature che nello scenario what-if non viene modificata insieme a NSmells. Il confronto diagnostico con NaiveBayes (In Proportion=6,62%, Sezione 3.3) supporta questa lettura, ma resta un'evidenza indiretta: non è stato condotto un esperimento controllato (es. rimuovendo dal training anche le feature correlate a NSmells) che la dimostri in modo conclusivo.
- **Assunzione di indipendenza tra smell e altre metriche.** L'intero scenario controfattuale (dataset B) assume che rimuovere gli smell non alteri le altre metriche della classe (dimensione, complessità, ecc.). È un'assunzione di comodo necessaria per costruire lo scenario, ma non realistica: nella pratica, rimuovere smell spesso comporta anche una riduzione di complessità o dimensione, che nello scenario simulato non viene modellata.

4.4 Milestone 4 – Refactoring automatico

- **Singolo strumento LLM.** Tutte le varianti sono state generate con un solo strumento (Microsoft Copilot, modalità Think Deeper). I risultati (in particolare l'identità tra C1 e C4, e il comportamento divergente di C3 su CommandLogic) sono specifici di questo strumento e non necessariamente generalizzabili ad altri LLM.
- **Discrepanza PMD/SonarCloud.** I due strumenti usano cataloghi di regole differenti (PMD in Milestone 1 per NSmells, SonarCloud in Milestone 4 per la diagnostica di refactoring): un confronto diretto tra "numero di smell rilevati" nei due contesti non è metodologicamente corretto, ed è per questo che le due fasi si sono basate su strumenti coerenti al proprio scopo, senza tentare una riconciliazione tra i due cataloghi.
- **Selezione delle classi.** I filtri applicati prima del ranking ($LOC \geq 60$, $5 \leq CC \leq 70$, almeno 2 metodi pubblici sostanziali, package *core/*) sono criteri progettuali non imposti dal corso: riflettono una scelta orientata a garantire materiale sufficiente per il refactoring, escludendo sia classi troppo piccole per offrire smell significativi sia classi con complessità estrema, la cui rifattorizzazione automatica sarebbe stata difficilmente valutabile in modo affidabile. Questi criteri introducono comunque un bias verso classi già relativamente sostanziose rispetto alla media del progetto, e la scelta specifica delle soglie resta arbitraria: soglie diverse avrebbero potuto restituire un ranking, e quindi una coppia di classi selezionate, differente.
- **Validazione del comportamento limitata alla capacità delle suite disponibili.** La regressione di C2-CommandLogic è stata rilevata solo grazie alle suite generate da LLM Few-shot e Zero-shot; suite manuali e Randoop non asserivano su quel percorso di codice. Ciò implica che eventuali regressioni in altre varianti (C1, C4, e tutte le varianti di NotificationDataBinderImpl), pur non rilevate da nessuna suite disponibile, non possono essere escluse con certezza – in particolare nelle porzioni di codice già note come strutturalmente irraggiungibili dalle suite di test (*read()* e *resolveReference()* di *CommandLogic*, *create()* di *NotificationDataBinderImpl* prima dell'introduzione della suite LLM).

- **C4-CommandLogic e le varianti C2-C4 di NotificationDataBinderImpl.** Poiché il codice sorgente refactored generato risulta identico a quello già testato, l'esito dei test non è stato rieseguito empiricamente per queste varianti, ma dedotto per equivalenza di codice: una scelta pragmatica per evitare esecuzioni ridondanti, che tuttavia non costituisce una verifica indipendente.
- **Assenza di EvoSuite tra le fonti di validazione.** La validazione comportamentale delle varianti si basa su tre fonti di test (manuale, Randoop, LLM), non quattro: EvoSuite è stato escluso per incompatibilità tecnica (Sezione 2.4.3), non per scelta metodologica. Non è quindi possibile escludere che test generati da EvoSuite – con la sua strategia di ricerca basata su algoritmi evolutivi, distinta sia da Randoop sia dagli LLM – avrebbero potuto rilevare regressioni comportamentali aggiuntive rispetto a quelle già trovate in *CommandLogic_C2*.

5. Conclusioni

Lo studio ha risposto alle quattro research question poste in apertura, integrando defect prediction, what-if analysis e refactoring automatico via LLM su Apache Syncope.

RQ1. Il dataset costruito (16.063 istanze, 6,56% di bugginess) conferma pattern noti in letteratura: le classi più difettose sono sistematicamente più grandi, più complesse e più accoppiate (FanOut, CyclomaticComplexity, NOM le metriche più correlate alla bugginess), mentre le metriche di processo legate al volume delle modifiche mostrano un potere predittivo più contenuto in questo progetto.

RQ2. RandomForest in configurazione baseline, senza feature selection né balancing, è risultato il classificatore più accurato, dominando su cinque delle sei metriche misurate (Precision, F-Measure, AUC, Kappa, NPofB20). L'unica eccezione è la Recall, dove le configurazioni con balancing ottengono risultati migliori al costo di una Precision molto più bassa – un compromesso classico tra le due metriche, non un'anomalia.

RQ3. La what-if analysis stima che fino al 37,95% delle predizioni buggy sul dataset completo sia riconducibile alla presenza di smell, un valore che l'analisi diagnostica con NaiveBayes suggerisce essere in parte amplificato da un'interazione tra NSmells e CyclomaticComplexity piuttosto che da un effetto puramente marginale della sola rimozione degli smell – un limite intrinseco di un modello, come RandomForest, capace di sfruttare interazioni non lineari tra feature correlate.

RQ4. Il refactoring automatico via LLM produce risultati eterogenei e dipendenti dalla ricchezza dello smell nella classe di partenza, più che dalla quantità di test forniti in input. Su NotificationDataBinderImpl, dove era presente un solo smell di natura strutturale (non legato a logica applicativa), tutte le varianti lo hanno rimosso in modo sicuro, restando identiche a C0 sia strutturalmente sia comportamentalmente. Su CommandLogic, dove lo smell era ripetuto cinque volte, il refactoring più aggressivo (C2, C3) ha rimosso tutti gli smell originari ma ha introdotto nuovi problemi – una regressione comportamentale in C2, rilevata solo dalle suite generate da LLM, e un peggioramento strutturale con fallimento di compilazione in C3. In entrambi i casi, fornire più contesto all'LLM (C4, con test BB+CF+MT) non ha prodotto un output diverso da quello ottenuto senza alcun test (C1): il modello ha generato lo stesso identico codice indipendentemente dal contesto disponibile.

Nel complesso, i risultati indicano che un refactoring generato da un LLM non garantisce, di per sé, un miglioramento della qualità del codice: la rimozione degli smell diagnosticati può introdurre regressioni comportamentali silenziose o nuovi problemi strutturali, rilevabili solo da una validazione multi-livello che combini test manuali, generati automaticamente (Randoop) e generati da LLM con tecniche diverse tra loro. Nessuna singola fonte di test, da sola, sarebbe stata sufficiente a intercettare tutti i problemi osservati in questo studio.

6. Takeaway messages

- **Un buon defect predictor non implica un buon "consulente di rimozione smell".** Il modello più accurato per predire la bugginess (RandomForest baseline) è anche quello più esposto a bias quando i suoi input sono metriche fortemente correlate tra loro (Sezione 3.3): usare un classificatore black-box per stimare l'impatto di un singolo intervento (rimuovere gli smell) richiede sempre un controllo diagnostico con un modello meno sensibile alle interazioni, prima di fidarsi del numero.

- **Più contesto in input a un LLM non garantisce un refactoring più sicuro.** Fornire suite di test progressivamente più ricche (da nessun test a BB+CF+Mutation) non ha cambiato l'output generato in nessuno dei due casi analizzati (C1=C4 byte-identiche): il fattore che ha determinato la qualità del refactoring non è stata la quantità di test disponibili, ma quante volte lo stesso smell si ripeteva nella classe originale.
- **Azzerare gli smell diagnosticati non equivale a migliorare il codice.** Le due varianti che hanno rimosso tutti gli smell segnalati da SonarCloud (C2, C3 su CommandLogic) lo hanno fatto applicando la correzione in modo uniforme e letterale, senza distinguere i casi in cui l'eccezione era effettivamente utilizzata: il risultato è stata una regressione comportamentale in un caso e un nuovo anti-pattern critico nell'altro. Un refactoring guidato solo da diagnostica statica, senza comprensione semantica del contesto applicativo, può risolvere il sintomo introducendo un problema diverso.
- **Nessuna singola fonte di test è sufficiente a validare un refactoring automatico.** La regressione di C2 è stata rilevata solo da suite generate da LLM con tecniche diverse (Few-shot, Zero-shot), non da test manuali né da Randoop. La validazione di codice generato da un LLM richiede una combinazione di fonti di test indipendenti tra loro – idealmente non generate dallo stesso modello che ha prodotto il codice da validare.

Riferimenti bibliografici

- [1] B. Vandehei, D. A. Da Costa e D. Falessi, «Leveraging the Defects Life Cycle to Label Affected Versions,» ACM Transactions on Software Engineering and Methodology, vol. 30, p. 35, 2021.
- [2] D. Falessi, S. Mesiano Laureani, J. Çarka, M. Esposito e D. A. Da Costa, «Enhancing the defectiveness prediction of methods and classes via JIT,» Empirical Software Engineering, vol. 28, n. 37, p. 43, 2023.
- [3] J. Çarka, D. Falessi e M. Esposito, «On effort-aware metrics for defect prediction,» Empirical Software Engineering, vol. 27, n. 152, p. 38, 2022.
- [4] D. Falessi, L. Narayana, J. F. Thai e B. Turhan, «Preserving Order of Data When Validating Defect Prediction Models,» p. 20.
- [5] D. Falessi, B. Russo e M. Kathleen, «What if I had no smells?,» ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, p. 17, 2017.

Disponibilità del materiale

Repository GitHub: https://github.com/valejin/Progetto_ISW2_BugginessPredictor

Dashboard SonarCloud: https://sonarcloud.io/project/overview?id=valejin_Progetto_ISW2_BugginessPredictor

Appendice A1: Tabelle e Figure

Tabella 1: distribuzione delle istanze classe-release e della bugginess per release

Indice	Release	Classi	Buggy	Non buggy	% buggy
1	1.0.0-incubating	488	12	476	2,46%
2	1.0.1-incubating	488	12	476	2,46%
3	1.0.3-incubating	489	21	468	4,29%
4	1.0.2-incubating	488	29	459	5,94%
5	1.0.4	489	31	458	6,34%
6	1.0.5	491	92	399	18,74%
7	1.0.6	498	68	430	13,65%
8	1.0.7	498	30	468	6,02%
9	1.1.0	645	64	581	9,92%
10	1.0.8	497	33	464	6,64%
11	1.1.1	646	59	587	9,13%
12	1.1.2	649	56	593	8,63%
13	1.1.3	653	84	569	12,86%
14	1.1.4	656	102	554	15,55%
15	1.1.5	668	88	580	13,17%
16	1.1.6	668	18	650	2,69%
17	1.1.7	670	43	627	6,42%
18	1.1.8	670	27	643	4,03%
19	1.2.0-M1	795	24	771	3,02%
20	1.2.0	812	44	768	5,42%
21	1.2.1	818	24	794	2,93%
22	1.2.2	819	18	801	2,20%
23	1.2.3	820	17	803	2,07%
24	1.2.4	821	35	786	4,26%
25	1.2.5	827	23	804	2,78%

Nota: l'indice segue l'ordine cronologico di rilascio usato nella pipeline, non l'ordinamento semantico del nome versione (si veda ad es. l'inversione tra le release 3 e 4).

Tabella 2: metriche raccolte nella Milestone 1

Metrica	Significato
Project	Nome del progetto software.
Release	Release software di riferimento.
ClassName	Nome della classe contenente il metodo, insieme al package a cui appartiene.
NOM	Numero di metodi dichiarati nella classe. Proxy diretto di dimensione e responsabilità della classe.
FanOut	Numero di classi interne da cui la classe dipende (import/uso). Misura l'accoppiamento in uscita: valori alti indicano una classe che delega/dipende da molte altre componenti.
CyclomaticComplexity	Complessità ciclomatica di McCabe: conta i percorsi di controllo indipendenti (if, cicli, branch). Valori alti indicano codice più difficile da comprendere e testare esaustivamente.
LOCTouched	Somma delle linee aggiunte e cancellate nei commit che hanno modificato la classe. Cattura l'entità complessiva delle modifiche subite.
NFix	Numero di fix commit (collegati a ticket di bug risolti) che hanno toccato la classe. Proxy diretto della storia dei difetti passati.
NAuth	Numero di autori distinti che hanno modificato la classe. Una forte dispersione di ownership è associata a minore coerenza nelle decisioni di design.
LOCAdded	Somma delle linee di codice aggiunte alla classe nei commit della finestra.
MaxLOCAdded	Massimo numero di linee aggiunte in un singolo commit. Cattura la presenza di modifiche "big bang" isolate.
Churn	Somma di linee aggiunte e cancellate (l'"agitazione" complessiva del codice). Alto churn è tradizionalmente associato a maggiore instabilità e rischio di difetti.
MaxChurn	Valore massimo di churn osservato in un singolo commit.
NS	Numero di sottosistemi (moduli di primo livello) toccati dai commit che hanno modificato la classe. Cambiamenti che attraversano più sottosistemi sono più a rischio.
ND	Numero di directory distinte toccate dai commit che hanno modificato la classe.
ChangeSetSize	Numero di file modificati insieme alla classe negli stessi commit. Change set ampi aumentano il rischio di interazioni non previste tra componenti.
Age	Età della classe, calcolata come giorni trascorsi dalla sua prima comparsa storica nel repository fino alla release considerata.
NSmells	Numero di code smell rilevati da PMD (ruleset quickstart.xml) sullo snapshot della classe in quella release.

Tabella 3: correlazione di ciascuna metrica con la label Bugginess (Pearson point-biserial e Spearman)

Metrica	Pearson r	Spearman ρ	Significativa ($p < 0,05$)
FanOut	0,325	0,259	Sì
CyclomaticComplexity	0,265	0,230	Sì
NOM	0,175	0,192	Sì
NAuth	0,165	0,151	Sì
NSmells	0,149	0,141	Sì
NFix	0,128	0,102	Sì
LOC Touched	0,013	0,061	Sì (ρ) / No (r)
ND	-0,013	0,061	Sì (ρ) / No (r)
NS	-0,004	0,061	Sì (ρ) / No (r)
MaxLOCAdded	0,008	0,060	Sì (ρ) / No (r)
LOCAdded	0,013	0,060	Sì (ρ) / No (r)
ChangeSetSize	-0,023	0,060	Sì
MaxChurn	0,004	0,039	Sì (ρ)
Churn	0,011	0,009	No
Age	-0,037	-0,032	Sì

Figura 1: distribuzione della classe Bugginess nel dataset (Weka Explorer, tab Preprocess)

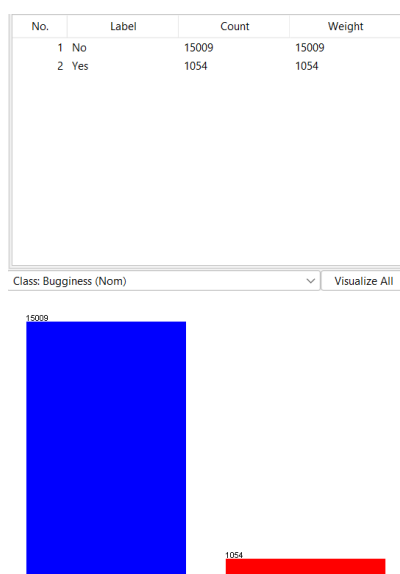
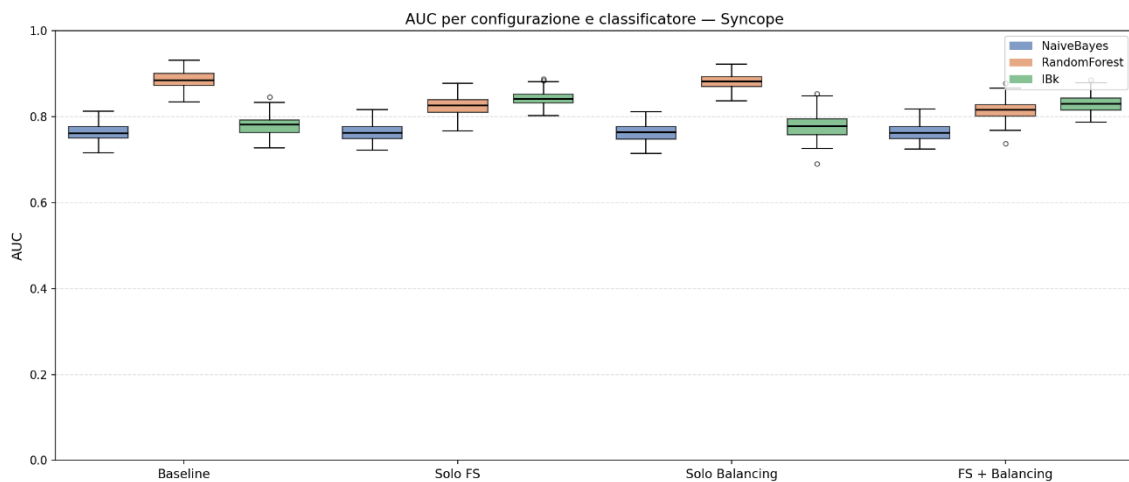
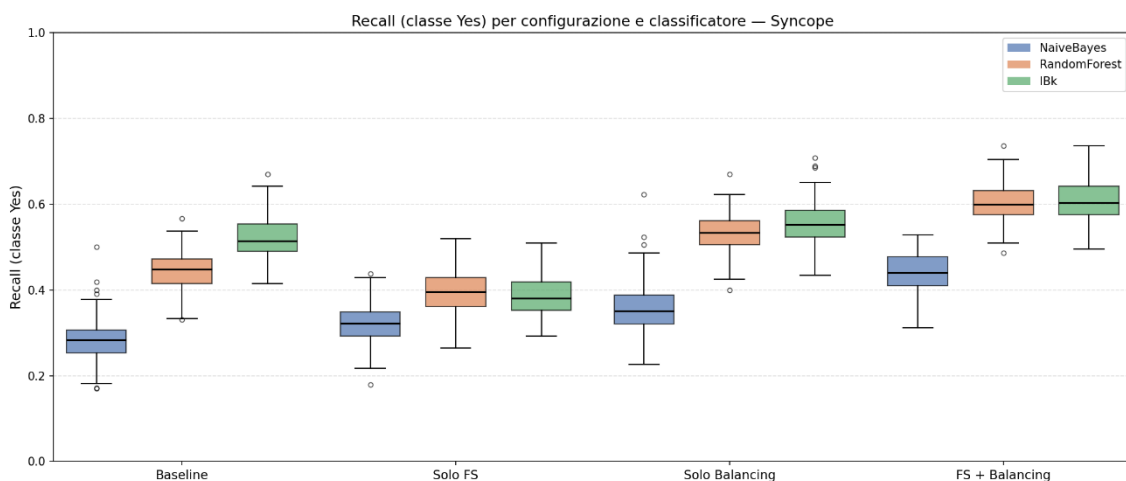
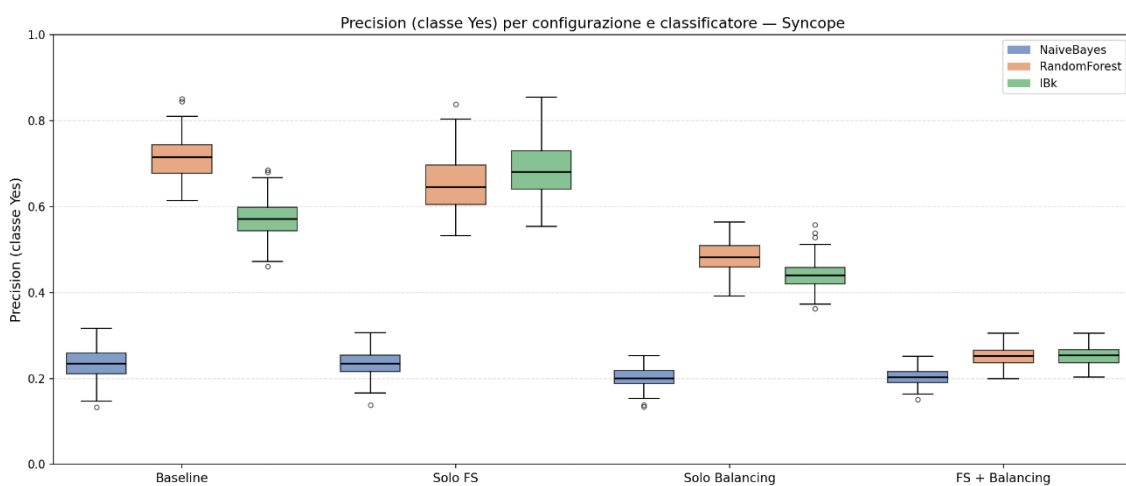
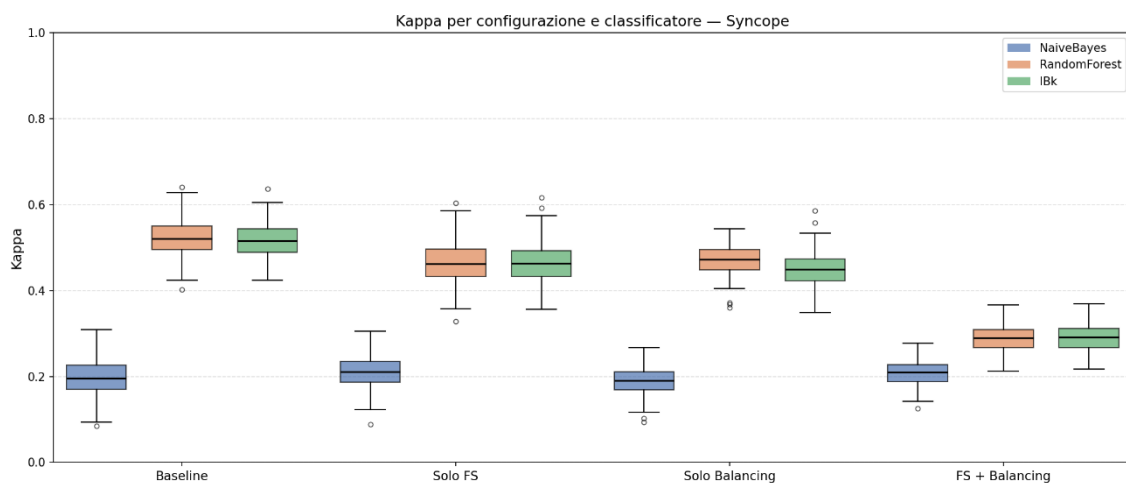


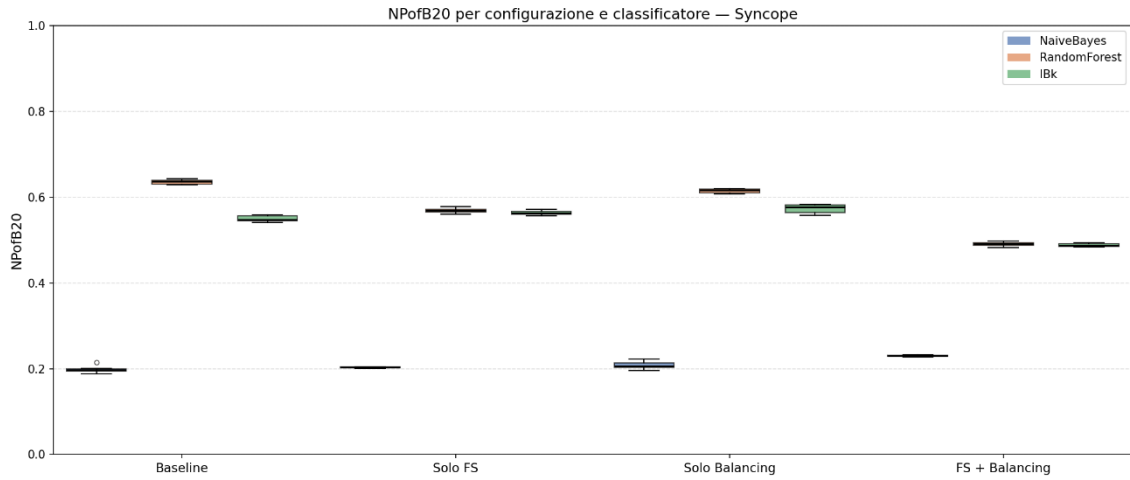
Tabella 4: risultati delle 12 configurazioni sperimentali (10×10-fold cross-validation, medie su 100 run)

Configurazione	Classificatore	Precision	Recall	F-Measure	AUC	Kappa	NPofB20
Baseline	RandomForest	0,7126	0,4452	0,5467	0,8858	0,5227	63,56%
Baseline	IBk	0,5709	0,5226	0,5446	0,7808	0,5142	54,94%
Baseline	NaiveBayes	0,2327	0,2810	0,2538	0,7638	0,1962	19,79%
Solo FS	RandomForest	0,6509	0,3952	0,4907	0,8254	0,4642	56,88%
Solo FS	IBk	0,6853	0,3849	0,4917	0,8423	0,4666	56,36%
Solo FS	NaiveBayes	0,2332	0,3191	0,2692	0,7632	0,2093	20,31%
Solo Balancing	RandomForest	0,4830	0,5328	0,5060	0,8826	0,4695	61,43%
Solo Balancing	IBk	0,4417	0,5557	0,4916	0,7785	0,4515	57,32%
Solo Balancing	NaiveBayes	0,2017	0,3604	0,2576	0,7626	0,1896	20,79%
FS + Balancing	RandomForest	0,2516	0,6033	0,3549	0,8159	0,2890	49,09%
FS + Balancing	IBk	0,2522	0,6095	0,3565	0,8299	0,2906	48,82%
FS + Balancing	NaiveBayes	0,2028	0,4432	0,2780	0,7632	0,2067	23,02%

Figura 2-7: distribuzione di AUC, Kappa, NPofB20, Precision e Recall per configurazione e classificatore







Dataset	Predetti Buggy	TN	FN	FP	TP
A	1.033	15006	24	3	1030
B+	607	5470	2	0	607
B	215	5461	403	9	206
C	426	9536	22	3	423

Tabella 5: matrice di confusione completa del BClassifier (RandomForest baseline) sui quattro sottoinsiemi della what-if analysis.

Classificatore	Buggy su A	Buggy su B+	Buggy su C	Buggy su B	In Total	In Proportion	Out of preventable
RandomForest (BClassifier)	1.033	607	426	215	392	37,95%	64,58%
NaiveBayes (controllo diagnostico)	1.254	1.024	230	941	83	6,62%	8,11%

Tabella 6: predizioni "buggy" sui quattro sottoinsiemi (A, B+, C, B) e metriche riassuntive della what-if analysis, per il BClassifier ufficiale e per il classificatore di controllo.

Metrica	Media A	Media B+	Media C	Media B (sint.)	Pearson r (NSmells)	Spearman ρ (NSmells)
CyclomaticComplexity	13,653	22,446	8,298	22,446	0,530*	0,438*
FanOut	4,360	6,960	2,776	6,960	0,442*	0,308*
NOM	6,536	8,211	5,516	8,211	0,285*	0,258*
NFix	0,622	1,119	0,320	1,119	0,242*	0,285*
NAuth	4,175	4,594	3,921	4,594	0,188*	0,178*
MaxLOCAdded	5,329	8,229	3,564	8,229	0,103*	0,150*
LOCAdded	12,352	21,312	6,896	21,312	0,104*	0,150*
LOC Touched	20,166	34,990	11,140	34,990	0,097*	0,148*
Churn	4,538	7,635	2,653	7,635	0,112*	0,057*
MaxChurn	3,916	6,455	2,370	6,455	0,092*	0,122*
NS	1,999	2,504	1,692	2,504	0,044*	0,146*
ND	14,634	17,150	13,102	17,150	0,029*	0,146*
ChangeSetSize	89,820	99,582	83,876	99,582	0,016*	0,145*
Age	924,905	846,672	972,539	846,672	-0,053*	-0,122*
NSmells	1,161	3,067	0,000	0,000	—	—

Tabella 7: media di ciascuna metrica sui quattro sottoinsiemi (A, B+, C, B) e correlazione con NSmells calcolata sul dataset completo (A). Asterisco = $p < 0,05$.

Classe	Variante	Smell originari residui	Nuovi smell introdotti	Delta netto vs C0
CommandLogic	C0	5	0	—
CommandLogic	C1	3	0	-2
CommandLogic	C2	0	2 (clumsy)	-3
CommandLogic	C3	0	1 (Critical, transazionale)	-4
CommandLogic	C4	3	0	-2 (identico a C1)
NotificationDataBinderImpl	C0	1	0	—
NotificationDataBinderImpl	C1-C4	0	0	-1

Tabella 8: numero di smell residui per variante rispetto a C0, con distinzione tra smell originari non rimossi e nuovi smell introdotti dal refactoring.

Classe	Varian te	SIZE_LO C	NO M	AVG_METHOD_SI ZE	CyclomaticComple xity	FanO ut
CommandLogic	C0	156	4	39,0	12	15
CommandLogic	C1	→	→	→	→	→
CommandLogic	C2	↓ 154(-2)	→	↓ 38,5	→	→
CommandLogic	C3	↑ 172(+16)	→	↑ 43,0	↑ 18 (+6)	→
CommandLogic	C4	→	→	→	→	→
NotificationDataBinderI mpl	C0-C4	184	3	61,3	12	15

Tabella 9: variazione delle metriche strutturali di ciascuna variante rispetto a C0 (→ = nessuna variazione).

Variante	BB+WB+MT	Randoop	LLM (ZS+FS+ToT)	Totale
C1	21/21	3/3	34/34	58/58
C2	21/21	3/3	30/34 (4 failure)	54/58
C3	—	—	—	non compila
C4	21/21	3/3	34/34	58/58

Tabella 10: esito dei test per variante, CommandLogic.

Variante	BB+WB+MT	Randoop	LLM	Totale
C1	24/24	3/3	34/34	61/61
C2	24/24	3/3	34/34	61/61
C3	24/24	3/3	34/34	61/61
C4	24/24	3/3	34/34	61/61

Tabella 11: esito dei test per variante, NotificationDataBinderImpl.

Appendice B1: Refactor prompt per LLM

You are an expert Java developer. I want to improve the maintainability of the attached C_0 class (<nome_classe>.java, from Apache Syncope, package org.apache.syncope.core.<...>).

Create C_1 without changing C_0 functionality and by removing the following smells, as reported by SonarCloud: <regole_violate_con_numero_di_riga_indicata>.

No test suite is provided as input for this variant (C_1); you must rely only on the class semantics and the smell description above to guide the refactoring.

Do not include in C_1 changes different from what I asked. In fact, consider that C_1 should replace C_0 and work with the other components of the system exactly as C_0 currently does, so keep method signatures and the public interface unchanged.

Return the entire refactored class as a single complete Java file, from the package declaration and imports down to the final closing brace, ready to replace C_0 as-is, not a diff, snippet, or partial excerpt.

This is an important request; take all the time you need to provide a complete and accurate answer to this request.

Testo prompt 1: Variante C1

You are an expert Java developer. I want to improve the maintainability of the attached C_0 class (<nome_classe>.java, from Apache Syncope, package org.apache.syncope.core.<...>).

Create C_2 without changing C_0 functionality and by removing the following smells, as reported by SonarCloud: <regole_violate_con_numero_di_riga_indicato>.

Make sure C_2 passes the attached Black Box test suite (<nome_classe>BBTest.java), which was designed with Category Partition and Boundary Value Analysis and is attached as a separate file. These tests must continue to pass unchanged against the refactored class.

Do not include in C_2 changes different from what I asked. In fact, consider that C_2 should replace C_0 and work with the other components of the system exactly as C_0 currently does, so keep method signatures and the public interface unchanged.

Return the entire refactored class as a single complete Java file, from the package declaration and imports down to the final closing brace, ready to replace C_0 as-is, not a diff, snippet, or partial excerpt.

This is an important request; take all the time you need to provide a complete and accurate answer to this request.

Testo prompt 2: Variante C2

You are an expert Java developer. I want to improve the maintainability of the attached C_0 class (<nome_classe>.java, from Apache Syncope, package org.apache.syncope.core.<...>).

Create C_3 without changing C_0 functionality and by removing the following smells, as reported by SonarCloud: <regole_violate_con_numero_di_riga_indicato>.

Make sure C_3 passes the attached Black Box test suite (<nome_classe>BBTest.java) and the attached Control Flow test suite (<nome_classe>WBTest.java, developed to improve statement and branch coverage using JaCoCo). Both suites are attached as separate files and must continue to pass unchanged against the refactored class.

Do not include in C_3 changes different from what I asked. In fact, consider that C_3 should replace C_0 and work with the other components of the system exactly as C_0 currently does, so keep method signatures and the public interface unchanged.

Return the entire refactored class as a single complete Java file, from the package declaration and imports down to the final closing brace, ready to replace C_0 as-is, not a diff, snippet, or partial excerpt.

This is an important request; take all the time you need to provide a complete and accurate answer to this request.

Testo prompt 3: Variante C3

You are an expert Java developer. I want to improve the maintainability of the attached C_0 class (<nome_classe>.java, from Apache Syncope, package org.apache.syncope.core.<...>).

Create C_4 without changing C_0 functionality and by removing the following smells, as reported by SonarCloud: <regole_violate_con_numero_di_riga_indicato>.

Make sure C_4 passes the attached Black Box test suite (<nome_classe>BBTest.java), the attached Control Flow test suite (<nome_classe>WBTest.java, developed to improve statement and branch coverage using JaCoCo), and the attached Mutation test suite (<nome_classe>MTTest.java, developed to improve mutation score using PIT). All three suites are attached as separate files and must continue to pass unchanged against the refactored class.

Do not include in C_4 changes different from what I asked. In fact, consider that C_4 should replace C_0 and work with the other components of the system exactly as C_0 currently does, so keep method signatures and the public interface unchanged.

Return the entire refactored class as a single complete Java file, from the package declaration and imports down to the final closing brace, ready to replace C_0 as-is, not a diff, snippet, or partial excerpt.

This is an important request; take all the time you need to provide a complete and accurate answer to this request.

C_0 source code: <codice_sorgente>

Testo prompt 4: Variante C4

Nota: A causa del limite di Copilot che supporta massimo tre file in allegato, si è scelto di incollare il codice sorgente della classe direttamente in prompt.